SESSION 15: Business and Management Data Processing I

WPM 15.3: ICON—A Management Information System

N. B. Solomon

Sylvania Electric Products Inc.

Needham, Mass.

FOR MANY YEARS, the word *mechanization* remained the key word in the computer solution of business problems. The objectives were confined to the realization of early benefits through reduction in the clerical working force. The resulting inefficient computer usage has led to a shift in emphasis from the autonomous computer program to an overall systems approach.

Within our division a project is under way directed towards the development of an integrated management control system for research and development projects. Known as the *ICON** information system it will provide both operational and executive levels of management with a comprehensive picture of a complex operation to facilitate decision making.

An initial analysis identifying various information levels is illustrated in Figure 1 *a* and *b*. Starting with the receipt of a request for quotation, data generation points are traced through the bid, award, and operational processes. The loop is then closed through customer reporting and management action.

Early efforts were devoted to the development of the planning phase, as shown in Figure 2. Initially, the system will generate time and dollar alternatives, independently, for each bid proposal. Eventually, a program will be developed to measure the effectiveness of all sets of alternatives. Optimization through reallocation of resources will represent the final objective.

The planning phase is centered around a two-dimensional array referred to as the task matrix; Figure 3. The matrix is a mapping of the complete contract showing both functional responsibility and organizational interface. It permits quantification and ordering of all task and organizational elements contributing to the particular contract under consideration.

* Integrated *CON*trol.

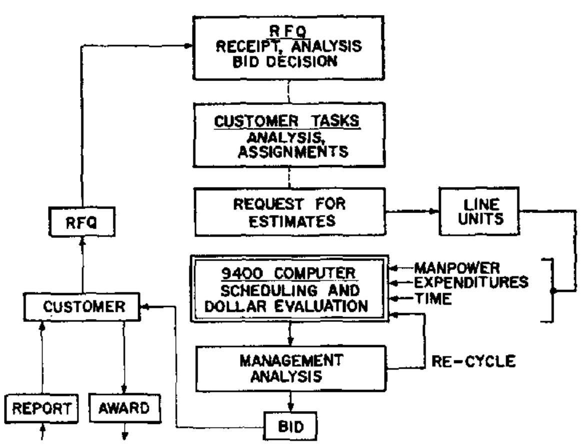
Technical preparation begins after matrix completion, through the expansion of each task matrix entry into a descriptive network (*PERT* array) of activities, events, and inter-relationships. In conjunction with this operation, a manpower and expenditure distribution is prepared for each activity in terms of labor classes and other elements of cost. Higher level *guide networks*, created with the task matrix, aid in network integration.

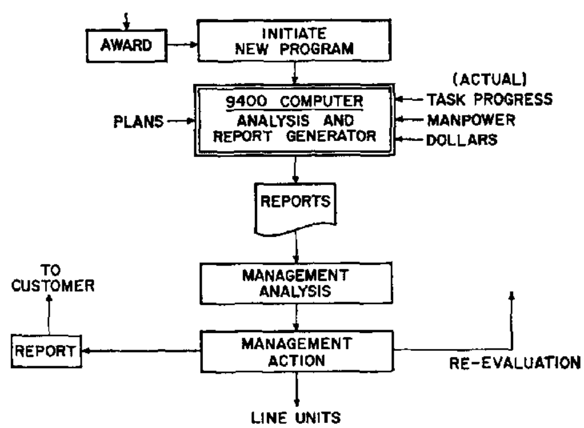
Computer processing of inputs derived from a *PERT* composite will determine schedules and path criticality. Recycling with modified parameters will be required to satisfy management objective dates. Schedules will then be established and the dollar evaluation phase begun.

With the acceptance of a *PERT* generated schedule, the time phased resource estimates are used to calculate direct labor and overhead costs; all elements-of-cost are then assembled. Multi-dimensional summarization, with reference to the task matrix, produces the estimated commitments, spread over the contract life, for each level within the customer task structure or within the internal organizational structure.

Modifications to the preliminary dollar evaluation may be recommended by management, based on the analysis of

[Continued on Page 125]





Figures 1*a* (left) and *b* (right)—Initial analysis. Drawing *a* illustrates how data-generation points are traced from the receipt of a *R*equest *F*or *Q*uotation through the planning phase to the formulation of the final bid proposal. The flow diagram *b* shows how, after a contract award, new information levels are created through actual contract operation and subsequent internal and customer reporting.

WAM 8.3: Control Techniques in the CL-11

[Continued from page 29]

(5)—Await I/O completions
(6)—Execute routines as tasks or paths
(7)—Establish new programs
(8)—End a path, etc.

The *awaiting* procedure is accomplished by marking a *PE* as not executable and chaining the *PE* into some mechanism which will, when the awaited condition is fulfilled, mark the *PE* as now being executable. As paths are formed, completed, postponed for an *await* condition, or interrupted from I/O completions, the status of the system changes and different paths are executed until programs are eventually completed. This causes the release of their *MA's*, thereby making room for other programs.

There exist two special tasks in the system. The first has the highest priority and performs various servicing functions involving file maintenance, operator negotiating, tape assigning, etc. The routines in this task are driven by queues of requests which are stored in the *Bucket Storage*. The second task has the lowest priority and resolves memory congestion problems by dumping or releasing reserved space.

Conclusion

The *CL-II Programming System* is believed to be a major advance in the utilization of present-day computers. The incorporation of a multi-program control system into an *extensible machine* has resulted in a more reasonable and natural vehicle for the implementation of many computer programs.

THAM 16.3: Function Evaluation

[Continued from page 63]

argument. This is the opposite of the previous case, as it should be since the square-root function is inverse to the square function.

Investigation of the base 2 exponential and logarithm functions yields a pair of adjustment rules both of which have $E(a) = 1$, and which together display the consistency appropriate to inverses. The adjustment of function values in these cases is uniquely specified by the conditions $m_2 = e_1 - 1$ for exponential and $e_2 = m_1 + 1$ for logarithm; in practice, the rules are subject to obvious modification when their strict application would lead to anomalies such as coefficient overflow or exponent underflow. Exponentials and logarithms to other bases are naturally obtained in terms of the base 2 versions by multiplying or dividing argument or function values by a constant; adjustment criteria are then appropriately determined by the composition process. Similar reasoning leads to adjustment rules for general power functions, via the composition $x^a = \exp(a \log x)$; the rules so derived are consistent with the square and square root rules cited earlier.

Satisfactory rules can be derived for other functions of common occurrence, although it is not generally feasible to optimize all the desired conditions simultaneously. The adjustment criteria so developed are applicable to the design of subroutines for evaluating functions in unnormalized arithmetic. They provide a useful estimate of the propagation of relative error, both for lengthy serial computations and for shorter ones where a high degree of functional composition is involved.

WPM 15.3: ICON

[Continued from page 59]

task and organizational summaries together with *critical-path* data and *uncertainty* factors. The task matrix manipulation program provides a means for operating on any of the data represented by the task matrix. Modifications can be applied to the significant cost segment, viz., the task matrix entry, or to any set of entries forming a task or organizational unit.

The integrated system for scheduling and dollar evaluation described for the project planning phase may be used after contract award for periodic dollar and time re-evaluation.

Report Phase

A report phase provides effective program control after contract award, through comparison of planned versus actual resources and progress; Figure 4b. Planning information, documented on magnetic tape, provides one of the basic inputs to this phase. Continuous source data collection, will make available all current transactions involving dollars, manpower, time, and task progress. Historical and cumulative information will provide the final input requirement.

The basic manpower and financial control reports generated by this phase for operational and executive management reflect the pyramidal structure of the task matrix and provide a realistic picture of task progress when combined with *PERT* scheduling reports. Special reports will be generated to satisfy the needs of decentralized operational management, where individual performance measurement indices exist.

Optimization

An advanced system aimed at maximization of overall company profits will be developed in three stages. The first program would consider each contract independently to measure the effectiveness of a time-cost balance based on the interaction of tasks. A second program would consider the re-distribution of resources across all contracts. The final stage of optimization would require, as input, past business history, present business operations, and general business policies.

Conclusion

Thorough task matrix representation and network derivation, management has available a complementary combination of control devices. The task matrix permits accountability by internal organizational unit or by tasks and sub-tasks.

System implementation in the planning phase has progressed to the point where complete dollar evaluation of a bid or contract in operation is now possible within twenty to thirty minutes of 9400 computer time. A modification, executed through the task matrix manipulation program, requires less than one minute. As a result, more time is now available for technical preparation and creation of alternate plans.

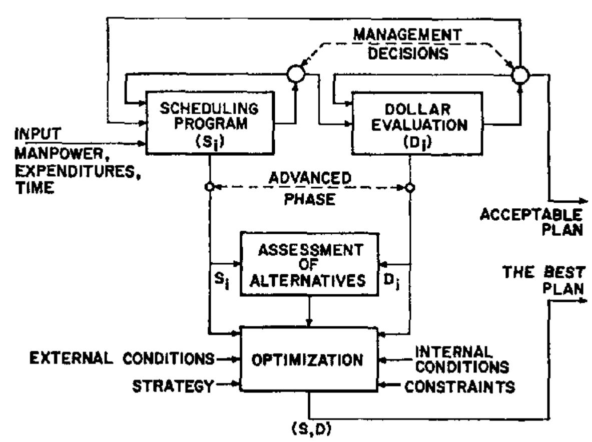


Figure 2—Planning phase—block diagram showing the major components of the planning phase. Included are two programs representing a proposed advanced phase.

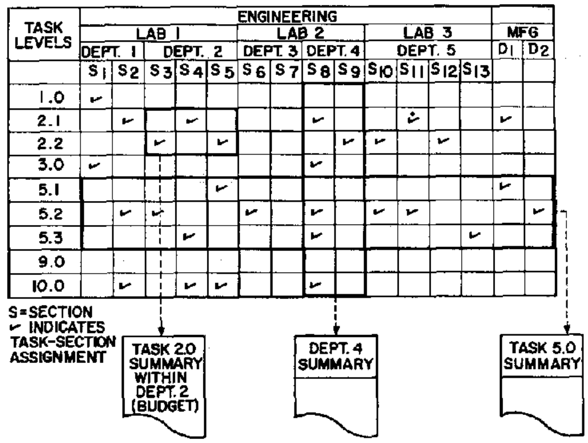
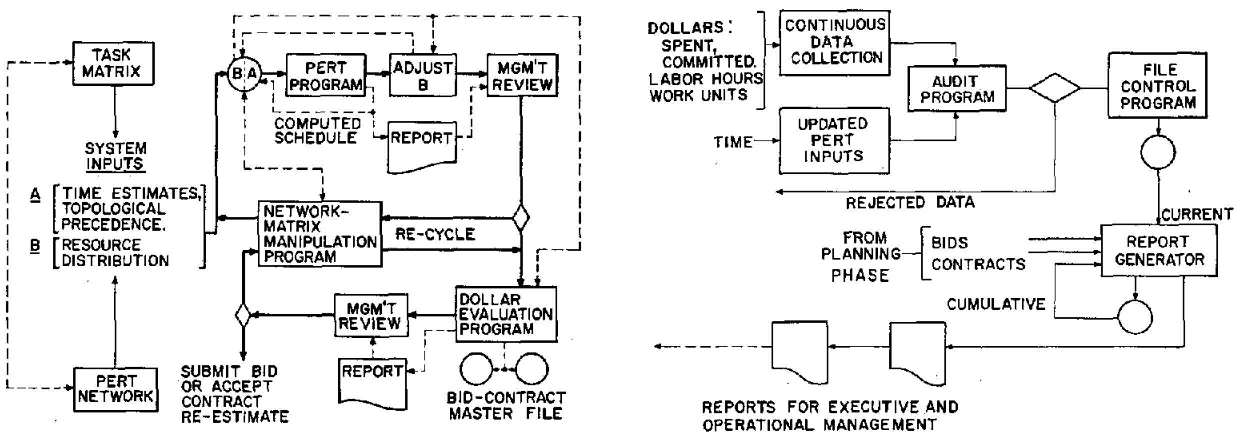


Figure 3 — A task matrix. A mapping of the contract identifying both organizational interface and functional responsibility through task-section assignments.



Figures 4*a* (left) and *b* (right)—The *ICON* system *(planning phase)* is illustrated at left. The scheduling loop is generally performed prior to dollar evaluation. Recycling through both loops will produce sets of alternatives expressed in dollars and time. Diagram at right shows the *report phase.* Actual costs incurred through contract operation are collected and processed against task progress and the budgetary information established during the planning phase.

WPM 12.2: History of Writing Compilers

[Continued from page 43]

a more elaborate scheme which needs more than a simple push-down list is required. There is even a generalization for $n = 0$, and on a zero-register machine the *IT* scheme is as good as any.

Many other improvements in object code efficiency are easily incorporated into the above algorithm, such as carrying a sign and absolute value tag with all operands, including temp storages.

Organization

The real differences between compilers is the way the basic components are organized into the program. The components themselves are essentially equivalent. The chief types of organization have been:

(1)—Symbol pair, used in *IT* and early compilers, in which each adjacent pair of symbols in a formula indicated what generator is to be used.

(2)—Operator pair, used in *NELIAC*, similar to the symbol-pair, but more economical since operands are treated more generally as a class by themselves.

(3)—Simple scan, similar to the algorithm given above, which generalizes operator-pair methods by putting operators into classes; they usually use several stacks instead of just one.

(4)—Recursive organization, where each generator can call on any other generator including itself, and delimiters are used to initiate and terminate control of each generator, as in the Carnegie Tech and the Burroughs B5000 *ALGOL* compilers.

(5)—Syntax-directed, analogous to recursively organized, except control is handled by one master generator which references syntax and semantics tables.

(6)—Various multiple pass compilers which have so complex an organization it cannot be summarized in less than several pages.